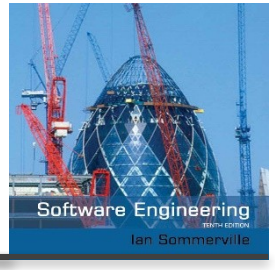
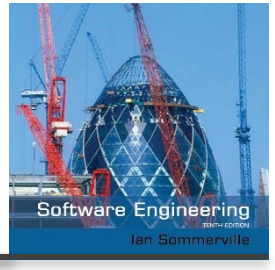


Chapter 3 – Agile Software Development

Rapid software development

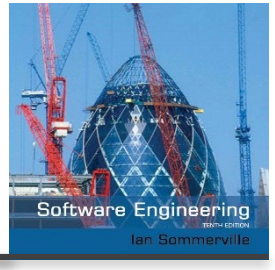


- **Business needs may change rapidly**

-> no stable requirements -> software has to change quickly to adapt these changes -> rapid development and delivery is the most important requirement for software systems

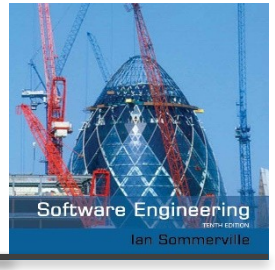
- **Requirements may change after software delivery** as the UX will help in improving design. Sometimes, external factors ,may lead to system change.
  - For example, a live streaming application with low bandwidth issue.

Rapid software development



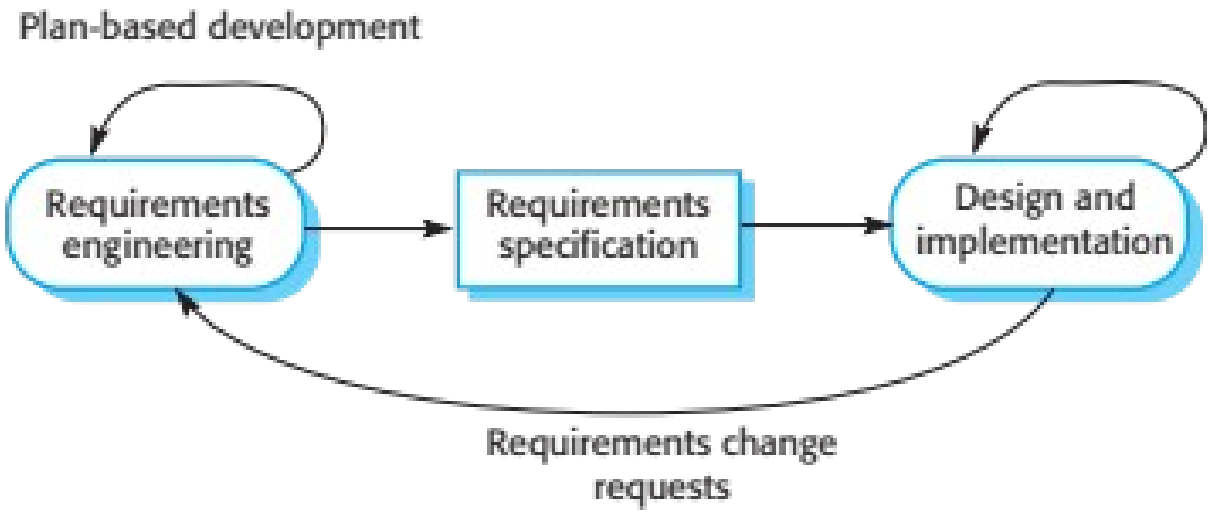
- Plan-driven development:
  - Could be helpful for some types of system but it **can not meet the rapidly changing business needs.**
  - **Can be helpful for some safety critical systems**
  - Using plan driven approach for software that have rapid changing business requirements might led to a delayed software product that would become obsolete after deployment.
- Agile development:
  - methods emerged in the late 1990s, whose aim was to radically **reduce the delivery time for working software systems**

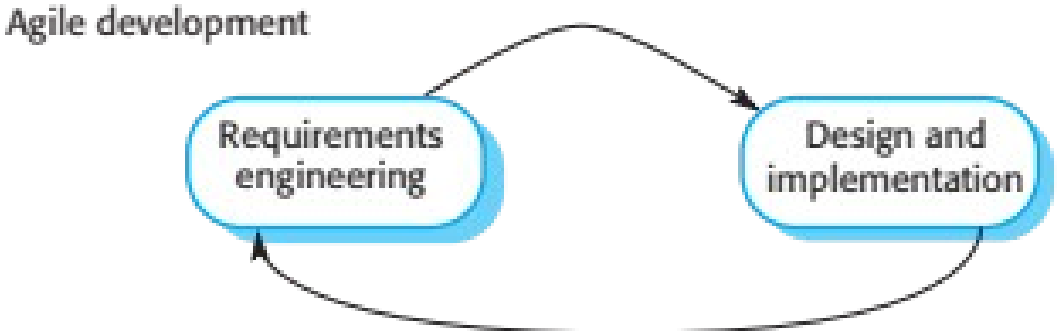
Agile development



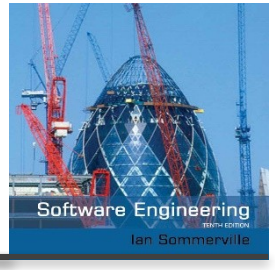
- **Program specification, design and implementation are inter-leaved**
- The system is **developed as a series of versions or increments with stakeholders involved in version specification and evaluation**
- **Frequent delivery of new versions for evaluation every 2-3 weeks**
- Extensive tool support to speedup the development process (e.g. automated testing tools, configuration/integration tools) e.g. selenium
- **Minimize documentation overhead**

Plan-driven and agile development comparison

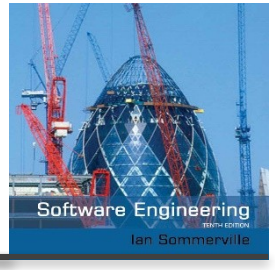




Agile development

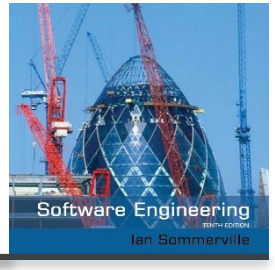


- Agile consider design & implementation as main activity.
- Requirement elicitation & testing are merged into the design & implementation phase.
- Issue?
  - We can’t keep track of the change in requirements as no official documents have been made
- So plan-driven / Agile ?
  - So a better approach is that the plan driven model may work in increments & agile methodology may focus a bit to documentation as well.



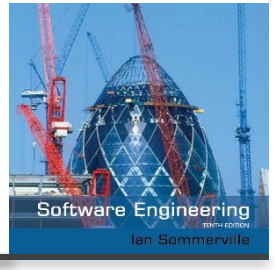
Agile methods

Agile methods

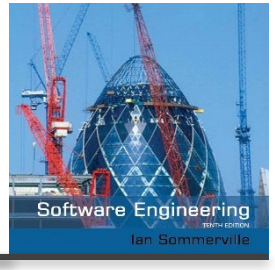


- Characteristics:
  - **Focus on the code rather than the design**
  - Are based on an **iterative approach to software development**
  - Are **intended to deliver working software quickly** and evolve this quickly to meet changing requirements.
- Aim:
  - **reduce overheads** in the software process (e.g. by **limiting documentation**)
  - **able to respond quickly to changing requirements** without excessive rework.

The principles of agile methods

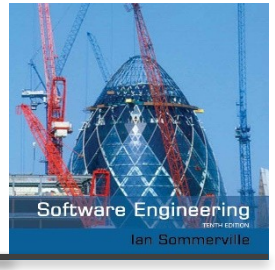


Principle	Description
Customer involvement	Customers should be closely involved throughout the development process. Their role is provide and prioritize new system requirements and to evaluate the iterations of the system.
Incremental delivery	The software is developed in increments with the customer specifying the requirements to be included in each increment.
People not process	The skills of the development team should be recognized and exploited. **Team members should be left to develop their own ways of working without prescriptive processes.**
Embrace change	Expect the system requirements to change and so design the system to accommodate these changes.
Maintain simplicity	**Focus on simplicity in both the software being developed and in the development process**. Wherever possible, actively work to eliminate complexity from the system.



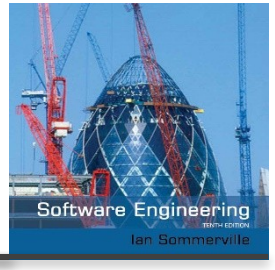
Agile development techniques

Extreme programming



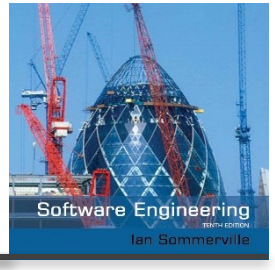
- Extreme Programming (XP) takes an ‘extreme’ approach to iterative development.
  - **Requirements are expressed as scenarios** (also user stories)
  - Deliver modules in **increments**;
  - All **tests must be run for every build** and the build is only accepted if tests run successfully.

Principles of Extreme programming



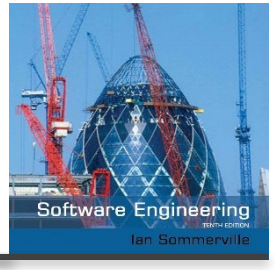
- Use **frequent iteration for developing User Stories.**
- **User stories**: simple and informal statements of the customer about the functionalities needed.
- On User stories, the project team proposes **Metaphors** (show how system would work?).
- The development team may decide to build a **Spike** for some feature.
- **Spike**: very simple program that is constructed to explore the suitability of the proposed solution (similar to prototype).

Extreme programming



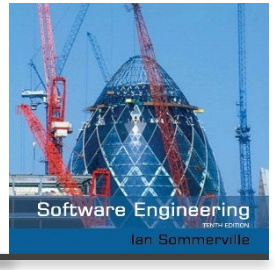
- Activities in XP:
  - listening
  - Designing
  - Coding
  - Testing
  - Feedback
- Applications that uses XP:
  - Small scale project
  - Project using new technology/ research based projects

Extreme programming practices (a)



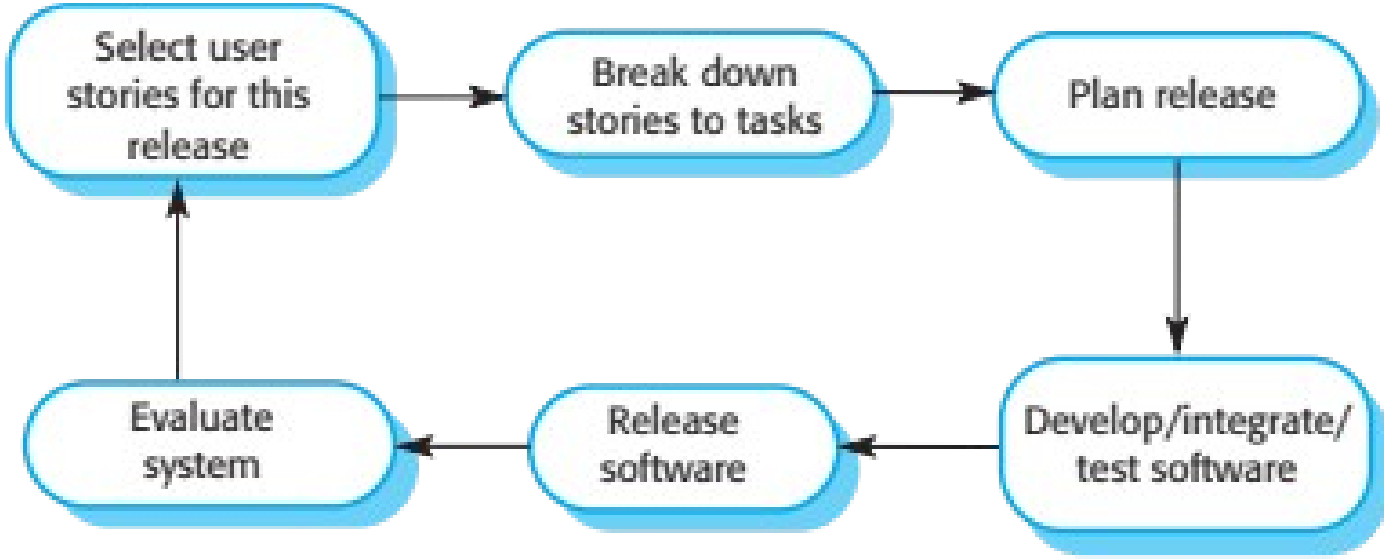
Principle or practice	Description
Incremental planning	• **Requirements are recorded on story cards** • **T**he stories to be included in a release are determined by the time available and their relative priority. • The developers break these stories into development ‘Tasks’.
Small releases	The minimal useful set of functionality is developed first. **Releases of the system are frequent** and incrementally add functionality to the first release.
Simple design	**Enough design is carried out to meet the current requirements and no more.**
Test-first development	An **automated unit test framework** is used **to write tests** for a new piece of functionality before that functionality itself is implemented.
Refactoring	All developers are expected to **refactor the code continuously** as soon as possible code improvements are found. This keeps the code simple and maintainable. e.g., a function may get deprecated.

Extreme programming practices (b)

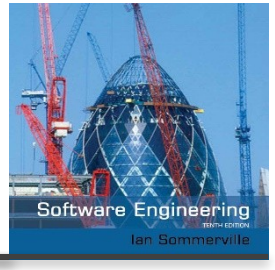


Pair programming	**Developers work in pairs**, checking each other's work and providing the support to always do a good job.
Collective ownership	The pairs of developers work on all areas of the system so that no islands of expertise develop, and **all the developers take responsibility for all the code**. Anyone can change anything.
Continuous integration	As soon as the work on a **task is complete**, it is **integrated into the whole system**. After any such integration, all the unit tests in the system must pass.
On-site customer	A representative of the end-user of the system (the **customer**) should be **available full time for the use of the XP team**. In an extreme programming process, the customer is a member of the development team and is responsible for bringing system requirements to the team for implementation.

The extreme programming release cycle

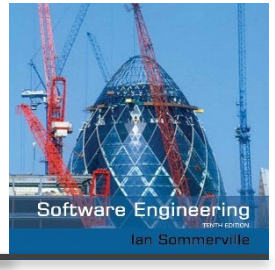


User stories for requirements



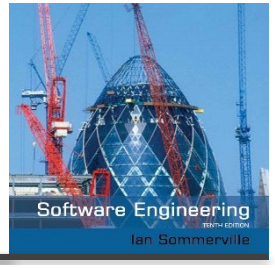
- User requirements are expressed as **user stories** or scenarios.
- **written on cards**; known as Task cards.
- Development team performs **tasks' breakdown based on deadline and cost estimates.**
- **The customer chooses the stories for inclusion in the next release based on their priorities and the schedule estimates.**

User stories for requirements

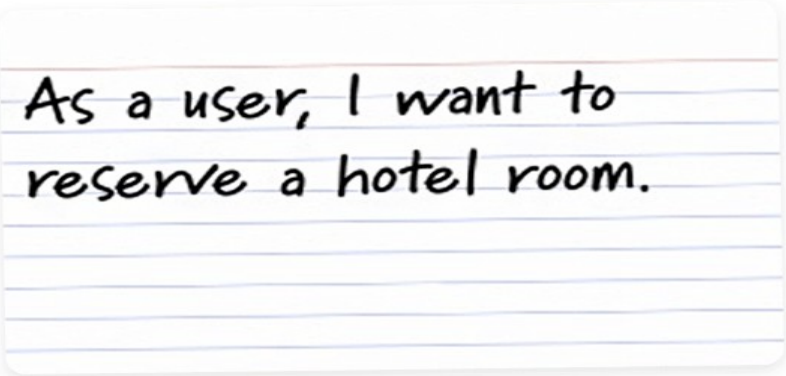


- This user stories idea is much helpful than working with traditional long requirement documents.
- Challenges:
  - Completeness.
  - Will the story cover all aspects of an activity?
  - Does The number of proposed user stories cover the whole system requirements?

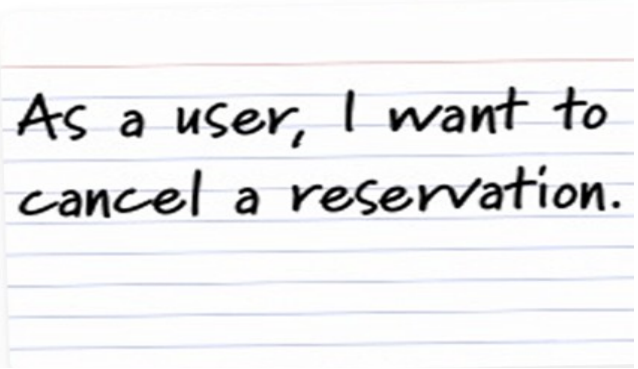
user stories for a travel website



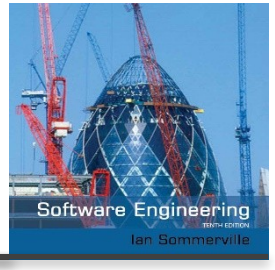
- Task 1:
  - Sign up
  - Login
  - Check for the availability
  - Reserve the room



- Task 2:
  - Cancel the reservation & pay penalty charges, if any
  - An email is sent to the Front Desk manager for room availability
  - A cancellation email is sent to the client.



task cards for the travel website?



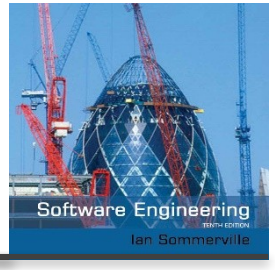
- Task 3:
  - go to the phot library
  - Select a hotel from the available options
  - Check for the photo dumps

As a vacation traveler, I want to see photos of the hotels.

- Task 4:

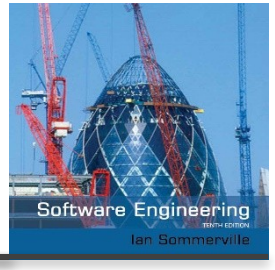
As a frequent flyer, I want to rebook a past trip so that I save time booking trips I take often.

Refactoring



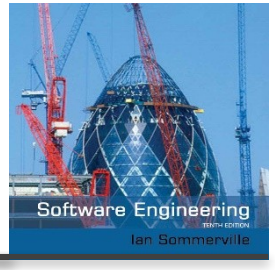
- Generally, it is practiced to develop a software keeping in Mind the **design for change approach**. It is worth spending time and effort anticipating changes as this reduces costs later in the life cycle.
- **XP doesn't support the idea** because it is possible that sometimes a code need to be changed utterly with a requirement change.
- Rather, it proposes **constant code improvement (refactoring)** to make changes easier when they needs to be implemented.

Refactoring



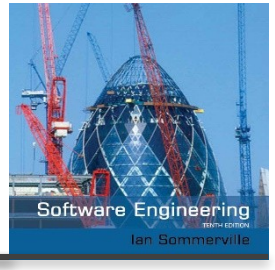
- Programming team **looks for possible software improvements**
- **make these improvements** even if there is no immediate need for them.
- This **improves the understandability of the software** and **so reduces the need for documentation.**
- **Changes are easier to make** because the code is well-structured and clear. (as **simplicity** is the essence of XP)

Examples of refactoring

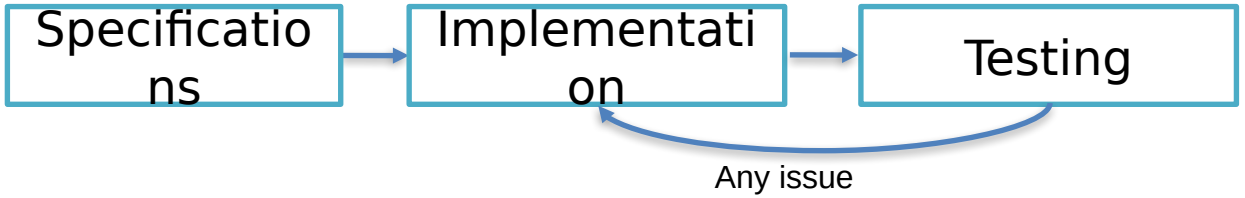


- Re-organization of a class hierarchy **to remove duplicate code.**
- Tidying up and **renaming attributes and methods** to make them easier to understand.
- The **replacement of inline code with calls to methods** that have been included in a program library.
- Generally, **IDEs include tools for refactoring**.
- Mostly, **refactoring is part of development** but when deadlines are hard and implementation is critical, then refactoring is **delayed**.

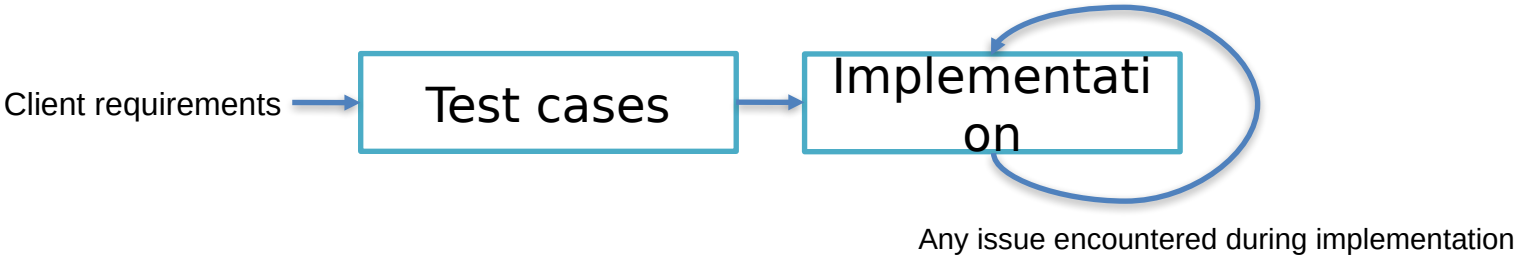
Test-first development (XP's innovation)



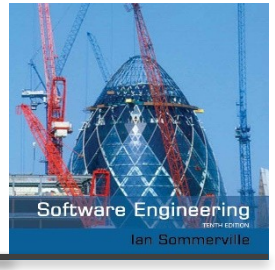
Plan driven approach



XP's approach (test first development)

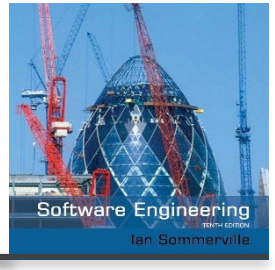


Test-first development



- XP testing advantages:
  - Problems of **requirements misunderstanding are reduced** as you have developed test cases as per your program's behaviors.
  - Avoids test-lag (when developer is faster than tester so pace mismatch)
  - **User involvement in test development and validation**.
  - **Tests are written as programs** rather than data so that they can be **executed automatically whenever a new functionality is added to a module**.

Customer involvement

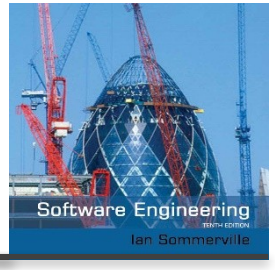


- ROLE:
  - to help **develop acceptance tests for the stories** that are **to be implemented in the next release** of the system.
  - If customer is a full-time member of development team, then he/she may **write testcases** as well.
- ISSUE:
  - **Customer have limited time**. So, he might be **reluctant to offer his services full time** with development team.

Test cases examples

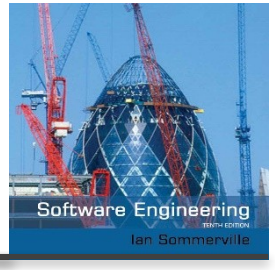
Pre-Conditions	Test Case	Test Steps	Test Results
<some Pre-condition>	Verify Login functionality with correct credentials	1. Navigate to <Enter URL Here>. 2. Enter Username : <Username>. 3. Enter Password: <Password> 4. Click on Submit button.	1. User navigates to Login Page. 2. User can enter username. 3. User can enter Password. 4. Home Page Opens.
<some Pre-condition>	Verify Login Functionality with incorrect username.	1. Navigate to <Enter URL Here>. 2. Enter Username : <Username>. 3. Enter Password: <Password> 4. Click on Submit button.	1. User navigates to Login Page. 2. User can enter username. 3. User can enter Password. 4. Error Message <Error Message Here> displayed.
<some Pre-condition>	Verify Login Functionality with correct username and incorrect password.	1. Navigate to <Enter URL Here>. 2. Enter Username : <Username> 3. Enter Password: <Password> 4. Click on Submit button.	1. User navigates to Login Page. 2. User can enter username. 3. User can enter Password. 4. Error Message <Error Message Here> displayed.

Test automation



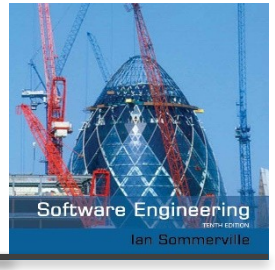
- **Tests are conducted using automated test tools**, they **simulate the program** depending upon the input and **check if the results meets the specifications**.
- A specific set of tests is quickly and easily executed whenever **any functionality is added to the system**, these automated test cases can be run, and problems in the new code can be identified immediately.

Problems with test-first development



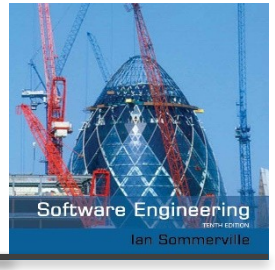
- **Programmers prefer programming to testing** and sometimes they take shortcuts when writing tests. For example, they may write **incomplete tests** that do not check for all possible exceptions that may occur.
- Some tests can be very difficult to write incrementally. **For example, in a complex user interface, it is often difficult to write unit tests for the code that implements the 'display logic' and workflow between screens**. As the design is changing continuously.
- It is difficult to judge the **completeness of a set of tests**. Although you may have a lot of system tests, **your test set may not provide complete coverage.**

Pair programming (XP's innovation)



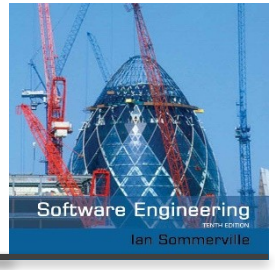
- Pair programming involves **programmers working in pairs**, and developing code together.
- This helps **develop collective ownership of code** and spreads knowledge across the team.
- It serves as an informal review process as **each line of code is looked at by more than 1 person**. Code implementation and inspection @ the same time.
- **A person doing refactoring is considered less efficient than the one doing coding**. So, this process **encourages refactoring** as **due to common ownership**, the whole team can benefit from the improved system code.

Pair programming

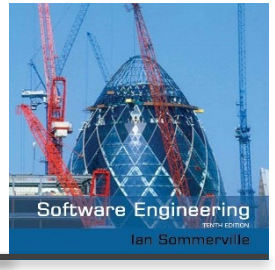


- In pair programming, **programmers sit together at the same computer to develop the software**.
- **Pairs are created dynamically** so that all team members work with each other during the development process.
- The **sharing of knowledge** that happens during pair programming is very important as it **reduces the overall risks to a project when team members leave**.
- Pair programming is **not necessarily inefficient** and there is some evidence that suggests that a pair working together is more efficient than 2 programmers working separately.

Pair programming

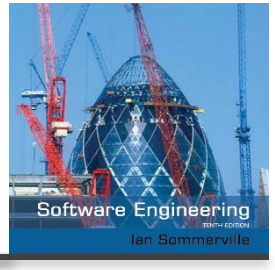


- Pair programming is not practiced mostly because mostly organizations think that two people working separately can do better than two doing in group.
- Solution?
  - Still, some of them believe it and practice it with slight modification in a way that in a pair of two, one is a senior developer, and the other one is a junior developer to support the **collaborative learning.**
  - **This shared learning reduces overall risks to project (in case a senior employee left).**



Agile project management

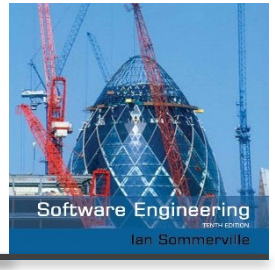
Agile project management



- Software **project managers focus** on:
  - **In-time** software delivery
  - and **within** the **planned budget**.

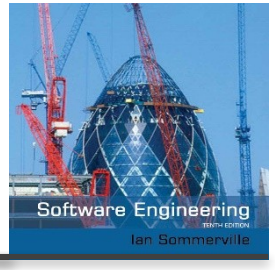
- **Agile project management** used an approach which is **adapted to incremental development** and the practices used in **agile methods**.

Scrum



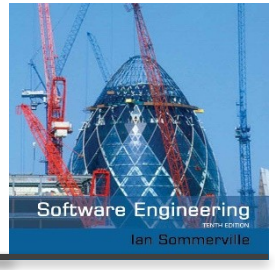
- Scrum is an agile method that provides a project management framework
- There are three phases in Scrum.
  - **The initial phase** is an outline planning phase where you establish the **general objectives for the project** and **design the software architecture**.
  - This is followed by a **series of sprint cycles,** where each cycle develops an increment of the system.
  - The **project closure phase wraps up the project**, completes required documentation such as system help frames and user manuals and assesses the lessons learned from the project.

Scrum terminology



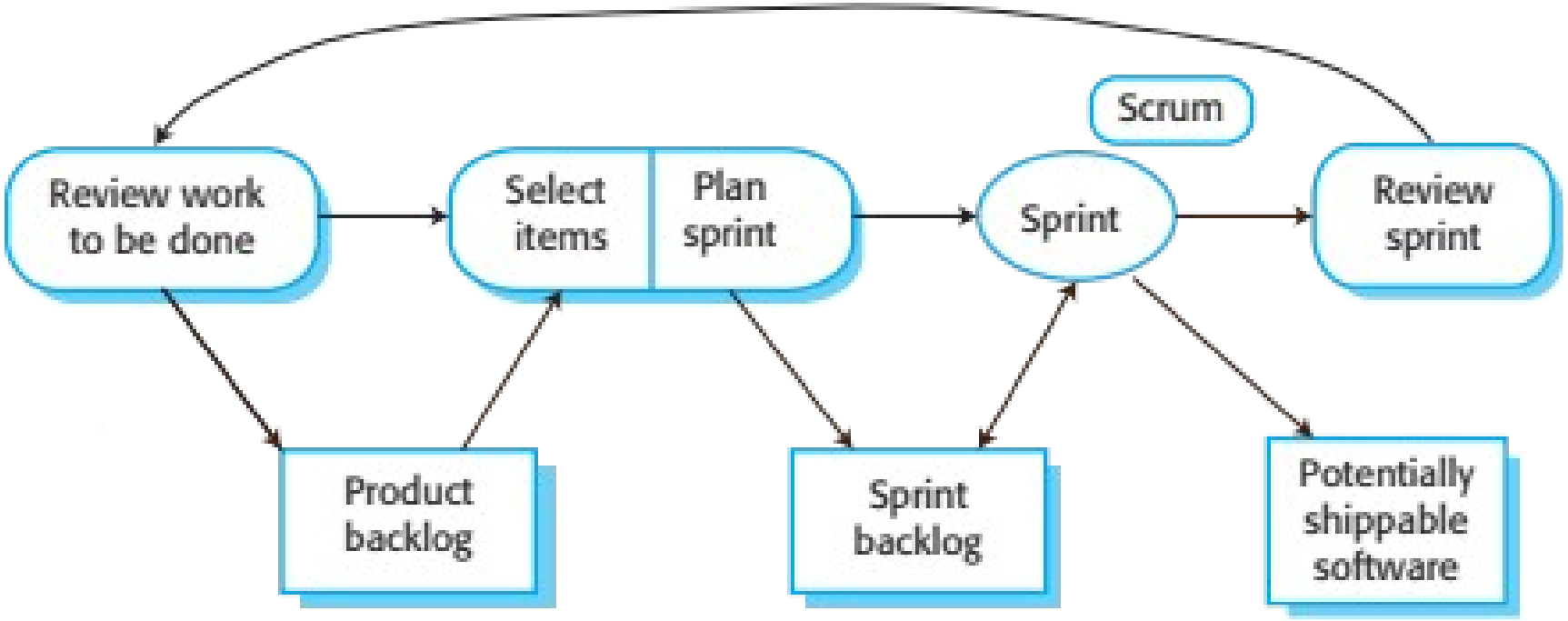
Scrum term	Definition
Development team	self-organizing group of software developers no more than 7 people. responsible for developing the software and essential project documents.
Sprint	**A development iteration.** usually, 2-4 weeks long.
Potentially shippable product increment	The software increment that is delivered from a sprint. **'potentially shippable' means it is in a finished state and no testing or any other work left to add it into the final product.**
Product backlog	**To-do list for Scrum Team.** It might be user stories or descriptions of supplementary tasks that are needed, such as architecture definition or user documentation.
Product owner	Identify requirements prioritize these for development continuously review the product backlog to ensure that the project continues to meet critical business needs. He/she could be a customer/product manager/a stakeholder representative.

Scrum terminology

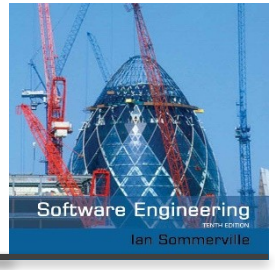


Scrum term	Definition
Scrum	**A daily meeting of the Scrum team** that reviews progress and prioritizes work to be done that day. Ideally, this should be a short face-to-face meeting that includes the whole team.
Scrum Master	Take follow up of work Guides team Deals with the organization and report team work. Similar to project manager.
Sprint Velocity	An estimate of **how much product backlog effort that a team can cover in a single sprint.** Understanding a team's velocity helps them estimate what can be covered in a sprint and provides a basis for performance improvement.

Scrum sprint cycle

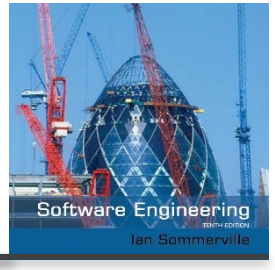


The Scrum sprint cycle



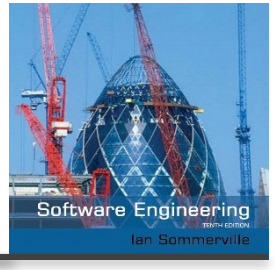
- **Sprints** are **fixed length**, normally **2–4 weeks**.
- The starting point for planning is the product backlog, which is the list of work to be done on the project.
- The **selection phase** involves the whole project team selecting the features and functionality from the product backlog to be developed during the sprint.

The Sprint cycle



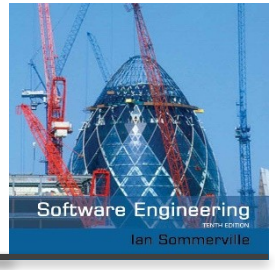
- Once these are agreed, the team organize themselves to develop the software.
- During this stage, the **team is isolated from the customer** and the organization, with all **communications channelled through** the so-called **‘Scrum master’.**
- The role of the **Scrum master is to protect the development team from external distractions**.
- At the **end of the sprint**, the **work done is reviewed** and **presented to stakeholders**. The next sprint cycle then begins.

Teamwork in Scrum



- ‘Scrum master’:
  - arranges daily meetings,
  - tracks the backlog of work to be done,
  - records decisions,
  - measures progress against the backlog
  - Communicates with customers and management outside of the team.
- The team:
  - attends short daily meetings (Scrums)
  - all team members share their progress, problems that have arisen and what is planned for the following day.
    - This means that everyone on the team knows what is going on and, if problems arise, can re-plan short-term work to cope with them.

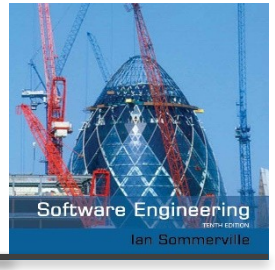
Scrum benefits



- The **product is broken down into a set of manageable** and understandable **chunks**.
- The **whole team have visibility of everything** and consequently team communication is improved.
- **Customers see on-time delivery of increments** and gain feedback on how the product works.
- **Trust between customers and developers is established** and a positive culture is created in which everyone expects the project to succeed.

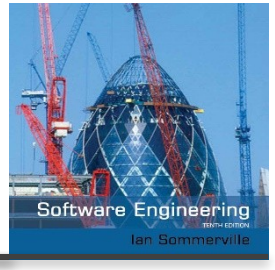
Scaling agile methods

Scaling agile methods



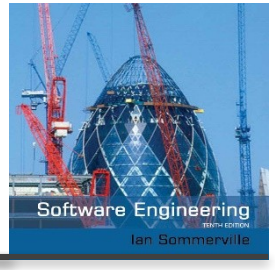
- **Agile methods are successful for small and medium sized projects that can be developed by a small co-located team.**
- Scaling Agile methods:
  - **the process of translating established Agile methods**, **like Scrum to larger groups of people.**
  - When scaling agile methods, it is important to maintain agile fundamentals:
    - Flexible planning,
    - frequent system releases,
    - continuous integration,
    - test-driven development
    - and good team communications.

Scaling out and scaling up



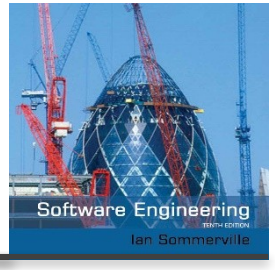
- ‘Scaling up’
  - **using agile methods for developing large software systems** that cannot be developed by a small team.
- ‘Scaling out’
  - **how agile methods can be introduced across a large organization** with many years of software development experience.

Practical problems with agile methods



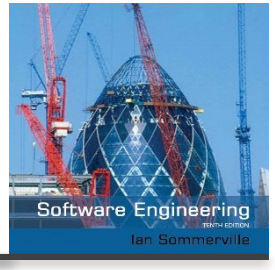
- **Problems:**
  - contract specifications in large organization.
  - **No documentations done so mostly appropriate for new software development rather than software maintenance**.
  - **designed for small software with generally co-located teams**

Contractual issues



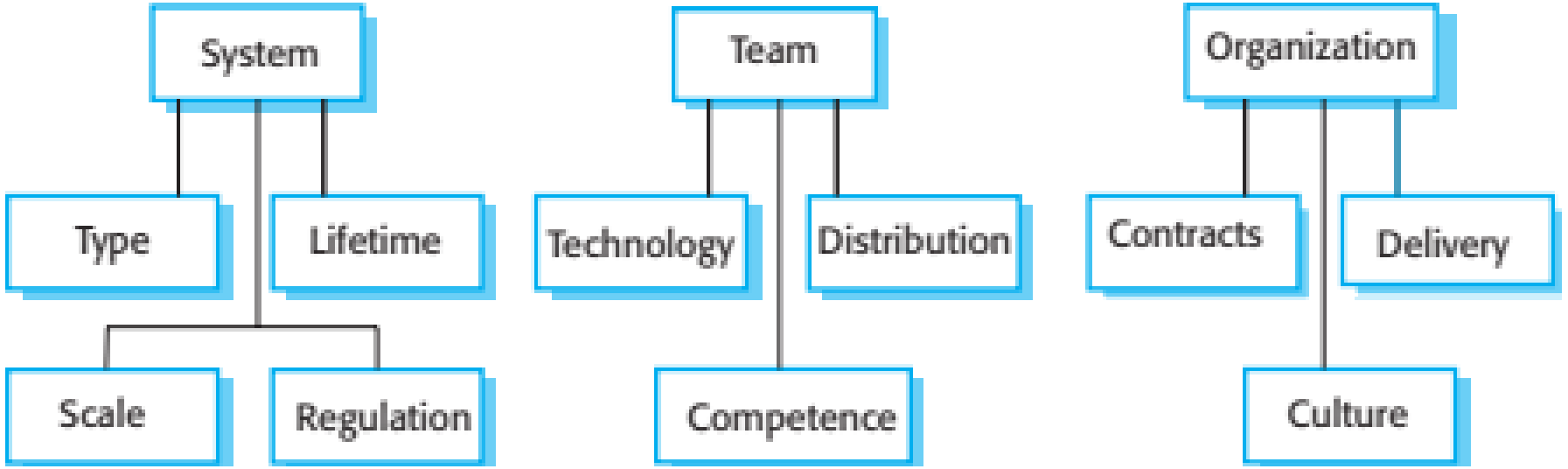
- Most software **contracts for custom systems** are based around a specification, which sets out **what has to be implemented by the system developer for the system customer.**
- However**, this precludes interleaving specification and development** as is the norm in agile development.
- **Solution:** A contract that pays for developer time rather than functionality is required

Agile methods and software maintenance

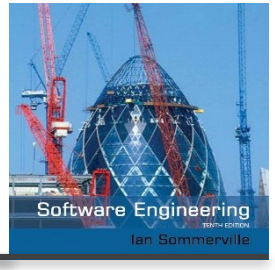


- **Most organizations spend more on maintaining existing software than they do on new software development**.
- So, for **agile methods** to be successful, they must **support maintenance as well as original development**.
- Key problems are:
  - **Lack of product documentation**
  - Keeping **customers involved** in the development process
  - Maintaining **the continuity of the development team**
- Agile development **relies on the development team knowing and understanding what must be done.**
- For **long-lifetime systems, this is a real problem as the original developers will not always work on the same system**.

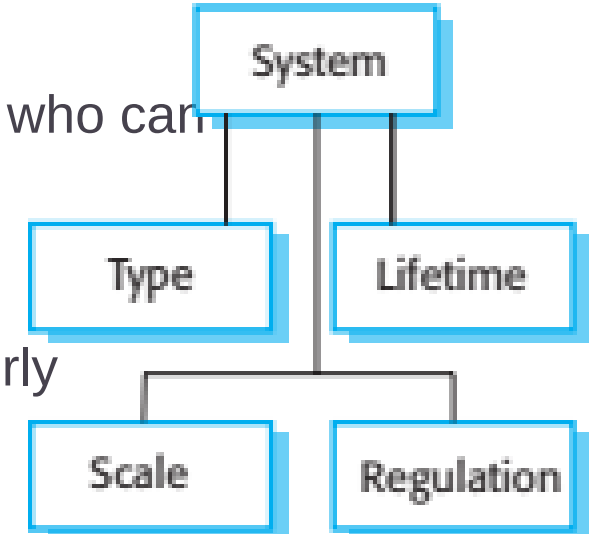
Agile important factors



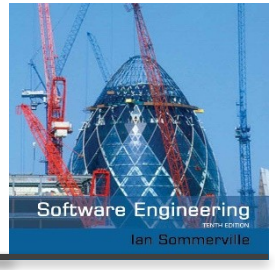
System issues



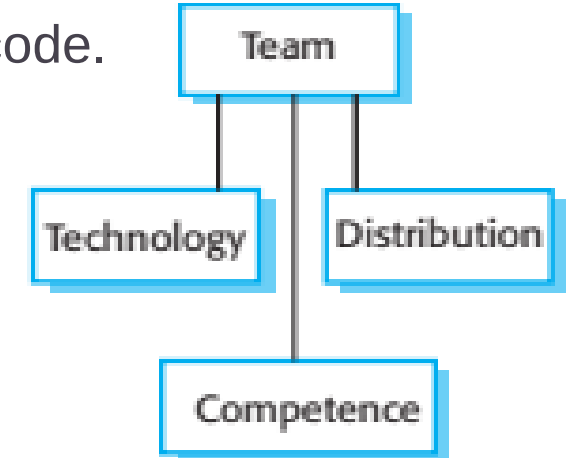
- **How large is the system** being developed?
  - Agile methods are most effective for a relatively small co-located team who can communicate informally.
- **What type of system** is being developed?
  - Systems that require a lot of analysis before implementation need a fairly detailed design to carry out this analysis.
- What is the **expected system lifetime**?
  - Long-lifetime systems require documentation to communicate the intentions of the system developers to the support team.
- Is the **system subject to external regulation**?
  - If a system is regulated you will probably be required to produce detailed documentation as part of the system safety case.



People and teams

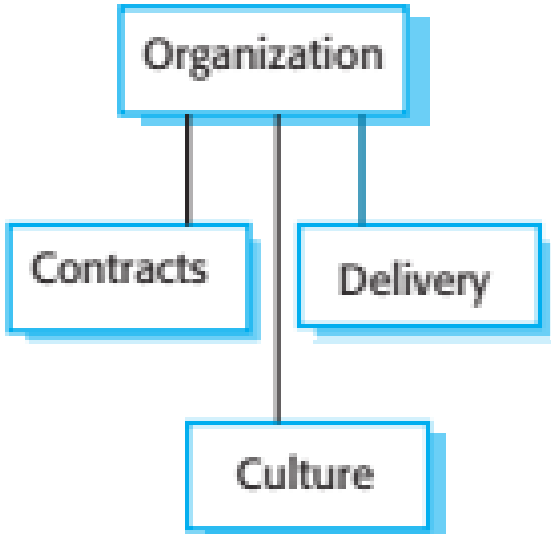


- How **good are the designers and programmers in the development team**?
  - It is sometimes argued that agile methods require higher skill levels than plan-based approaches in which programmers simply translate a detailed design into code.
- How is the **development team organized**?
  - Co-located/distributed
  - Design documents may be required if the team is distributed.
- What **support technologies are available**?
  - IDE support for visualisation and program analysis is essential if design documentation is not available.

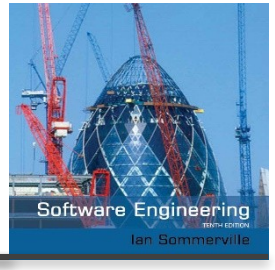


Organizational issues

- What is the work culture either plan-based or agile
- How the contracts have been designed for the maintenance phase
- Will customer representatives be available to provide feedback of system increments?

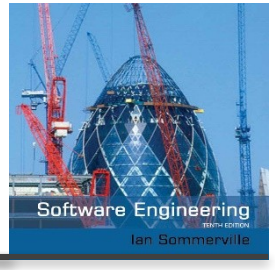


Agile methods for large systems



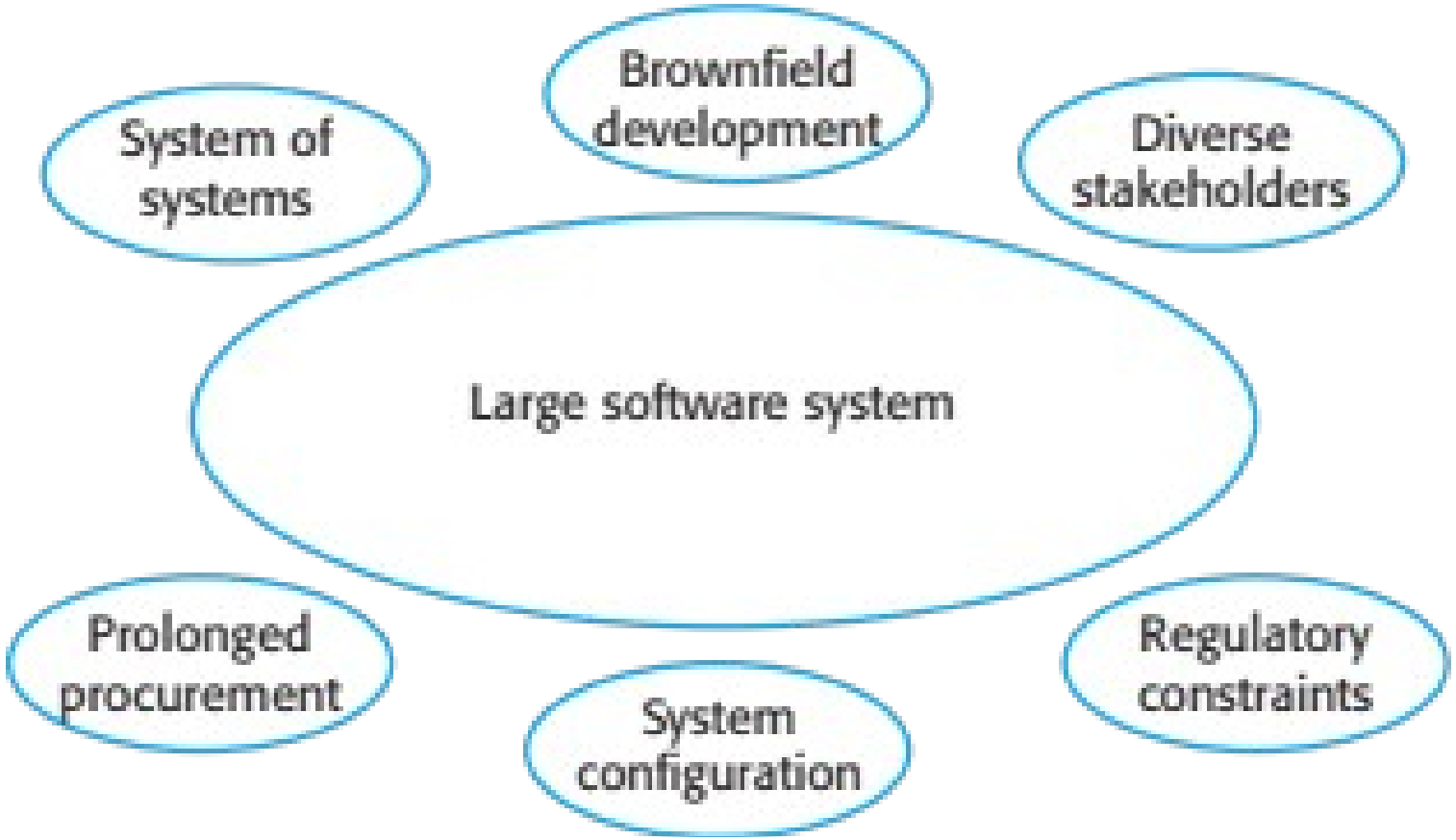
- Large systems-> larger teams in different zones may be
- **Large systems are ‘brownfield systems’,(the systems derived from an existing system)**. Many of the system requirements are concerned with this interaction and so don’t really lend themselves to **flexibility and incremental development.**
- When existing systems are integrated -> a lot of time is wasted on integration rather than coding.

Large system development

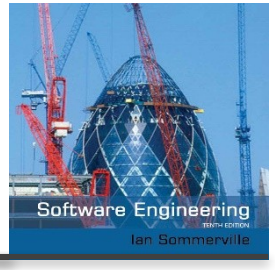


- **Large systems and their development processes are often constrained by external rules and regulations** limiting the way that they can be developed.
- Large systems **have a long procurement and development time**. Its really difficult to hold people for a project.
- Large systems have diverse stakeholders-> so its impossible to involve all of them in the development process.

Factors in large systems

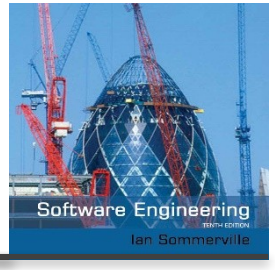


Scaling up to large systems



- A **completely incremental** approach to requirements engineering **is impossible.**
- Diverse and multiple product owners or customer representatives.
- For large systems development, it is not possible to focus only on the code of the system. As procurement and integration procedure is also involved
- Cross-team communication mechanisms have to be designed and used.
- Continuous integration is practically impossible. However, it is essential to maintain frequent system builds and regular releases of the system.

Multi-team Scrum



- *Role replication*
  - **Each team has a Product Owner for their work component and ScrumMaster**.
- *Product architects*
  - **Each team chooses a product architect and these architects collaborate to design and evolve the overall system architecture.**
- *Release alignment*
  - **The dates of product releases from each team are aligned** so that a demonstrable and complete system is produced.
- *Scrum of Scrums*
  - **There is a daily Scrum of Scrums where representatives from each team meet to discuss progress and plan work to be done**.